



29

Orchestration Prefect (design)

US3.4 · C6/C7

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Justifier l'orchestration (retries, scheduling, observabilité)
- › Distinguer task et flow ; raisonner idempotence
- › Concevoir le flow ingest→feature→predict→store
- › Repérer la fuite inter-machines (merge_asof sans by)

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE flow hello exécutable (retries) · design (mermaid + table I/O)

1

Module 29 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Toutes les heures, il faut ingérer les nouvelles mesures, prédire, puis stocker. À la main c'est impensable : on conçoit un flow qui s'exécute seul et se reprend en cas d'erreur.

Angle éthique / risque (C2) Une automatisation sans garde-fou propage une erreur à grande échelle. Retry et alerting bornent le risque.

■ Contexte & outils — pourquoi ce module

Contexte : lancer le pipeline à la main n'est pas fiable. Pourquoi : on l'AUTOMATISE et on le rend rejouable sans dégâts.

- Prefect : orchestrateur de flux (@flow / @task, planification)
- Idempotence : rejouer un run ne crée pas de doublon
- Retry + backoff : encaisse les pannes passagères
- merge_asof by="machine" : jointure temporelle SANS fuite entre machines

Objectif du module Un flow ingest -> predict -> store, fiable et relançable.

■ Pourquoi orchestrer (et pas juste cron)

Un orchestrateur ajoute à cron les réessais, l'observabilité et les dépendances entre étapes.



i

Le jargon décodé

Task = une étape. Flow = l'orchestration des tasks. Backoff = délai croissant entre réessais. Idempotent = rejouer ne change rien.

≈

L'analogie

cron est un réveil (il sonne à l'heure). Un orchestrateur est un chef de gare : il enchaîne les trains, gère retards et correspondances.

EN UNE PHRASE Orchestrer = rendre le pipeline résilient ET observable, pas juste le lancer.

■ Idempotence & retries : rejouer sans casser

On ne réessaie que les erreurs transitoires, et chaque étape doit pouvoir être rejouée sans effet de bord.

- Transitoire (DB indisponible 1 s) → retry ; déterministe (schéma invalide) → échouera toujours
- Idempotence = condition d'une reprise sûre après incident

+ Astuce de pro

Figiez les contrats des tasks (entrées/sorties/erreurs) AVANT de coder : on évite de découvrir les incompatibilités à l'exécution.

? Idée reçue

« Il faut réessayer toutes les erreurs. » FAUX : réessayer une erreur déterministe = boucle inutile. Retry = transitoire uniquement.

EN UNE PHRASE Retry sur le transitoire ; idempotence pour la reprise.

■ Le piège data : la fuite inter-machines

Joindre température et pression sans préciser la machine fait hériter à une machine la pression d'une autre.

- merge_asof sans by="machine" : MACH-01 récupère la pression de MACH-02 au même instant
- Correction : by="machine" (tolérance ± 90 min, nearest) ; résidu non-joint $\approx 1,76$ %

! Attention — piège

merge_asof sans by ne lève AUCUNE erreur : le bug est silencieux, visible seulement si on inspecte les valeurs.

→ Cas réel InduSense

Symptôme : pressions incohérentes pour une machine → hypothèse → by="machine" → test qui repasse au vert.

EN UNE PHRASE by="machine" : la clé qui empêche la fuite inter-machines.

■ Le flow, en vrai (task / flow + la jointure)

Le flow orchestre des tasks ; la task ingest joint température et pression PAR machine.

```
@task(retries=2, retry_delay_seconds=3)
def ingest(data_dir):
    temp = temp.sort_values("timestamp")
    pres = pres.sort_values("timestamp")
    return pd.merge_asof(temp, pres, on="timestamp", by="machine",
                        direction="nearest", tolerance=pd.Timedelta(minutes=90))

@flow
def predict_flow(data_dir):
    return store(predict(feature(ingest(data_dir))))
```

! Attention — piège

merge_asof SANS by="machine" : une machine hérite de la pression d'une autre (fuite silencieuse, 0 erreur).

+ Astuce de pro

Retries sur le transitoire (DB 1 s), jamais sur une erreur déterministe (schéma).

EN UNE PHRASE Orchestrer = retries + observabilité + dépendances. C6/C7.

■ À retenir en 3 points

- Un flow enchaîne des tâches.
- Retry + backoff = robustesse face aux pannes transitoires.
- On conçoit l'idempotence AVANT de coder.

2

Module 29 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — flow « hello »

```
@task(retries=2, retry_delay_seconds=3)
def ping(name): return f"pong:{name}"
@flow
def hello_flow(name="indusense"): return ping(name)
```

✓ Point de contrôle `uv run python -m indusense.flows.hello` → `pong:indusense` + run loggé.

■ TP — design du flow

```
ingest → feature → predict → store  
(table I/O : entrée / sortie / erreur, par tâche)
```

Données réelles ingest = merge_asof by="machine" (± 90 min) ; résidu non-joint $\approx 1,76$ %.

3

Module 29 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — paramétrer le flow

Avancez à votre rythme. Le formateur reste disponible.

- Ajouter un paramètre (@flow recevant data_dir) + logs par task
- OU dessiner une branche d'alerte (si proba > seuil)

Posture en distanciel

On valide le contrat des tâches avant d'implémenter (module 30). Partagez votre diagramme pour relecture express.

■ Definition of Done & transition

- DoD : flow hello exécutable + design (mermaid + table I/O)

Transition → Module 30 On implémente le flow réel sur les vraies données + persistance.